



University of Information Technology and Sciences

Department of Computer Science and Engineering

LAB ASSIGNMENT

Solving Four LeetCode Problems Using C++

Course Name : Data Structures and Algorithms I Lab

Course Code : CSE0613212

Course Session : Autumn 2026

SUBMITTED TO

Md. Yousuf Ali

Lecturer, Course Teacher

SUBMITTED BY

Gazi Forhad Hossain

Student ID: 04325205101015

Batch: 58

Section: A1

Contents and Learning Objectives

This assignment solves four common DSA problems using simple C++ logic. Each solution includes comments, a sample run, an algorithm summary, and time/space complexity.

Problems included

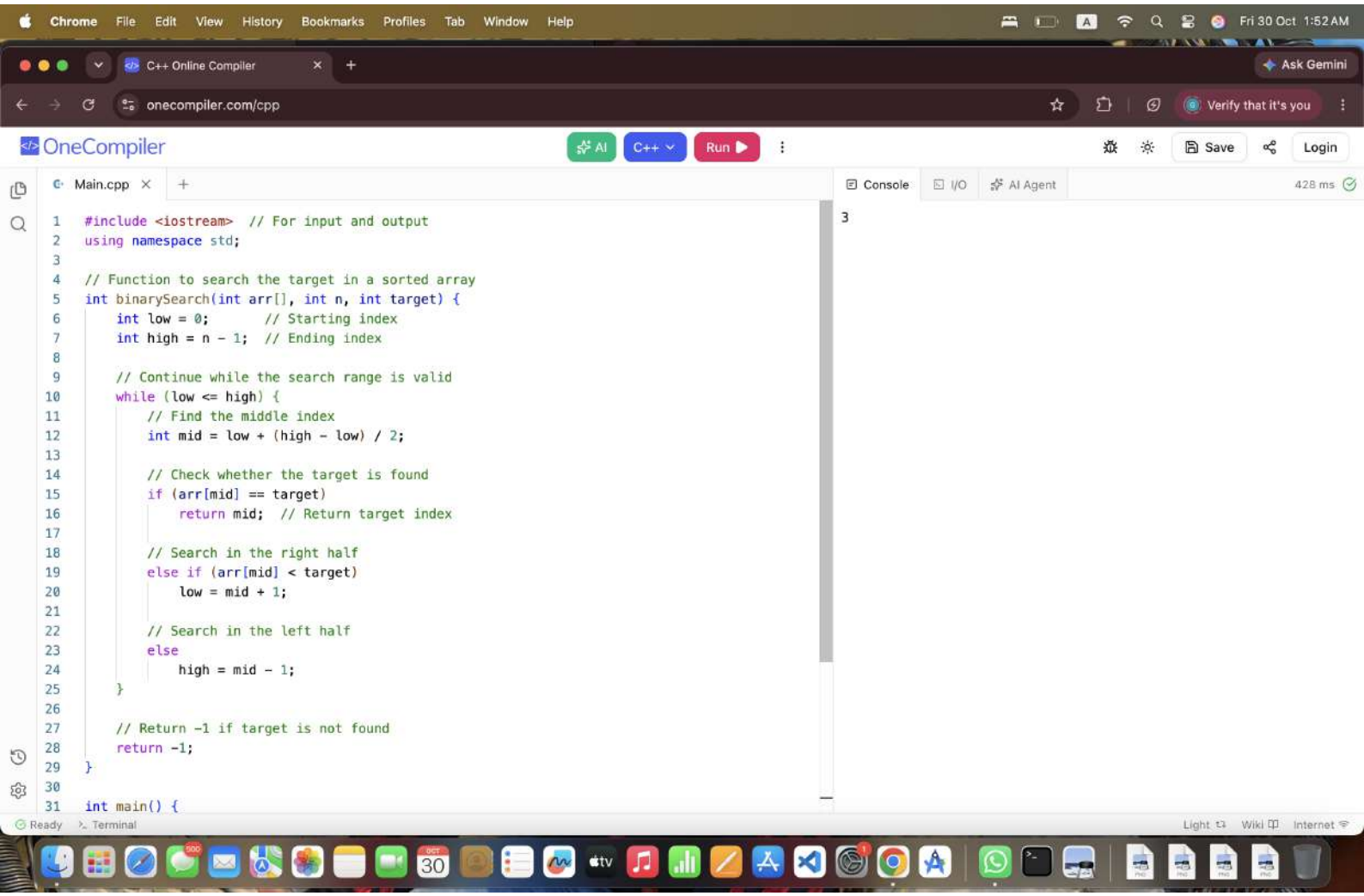
- Q1. Binary Search in a sorted array
- Q2. Reverse a singly linked list
- Q3. Check whether a matrix is a Toeplitz matrix
- Q4. Validate brackets using a stack

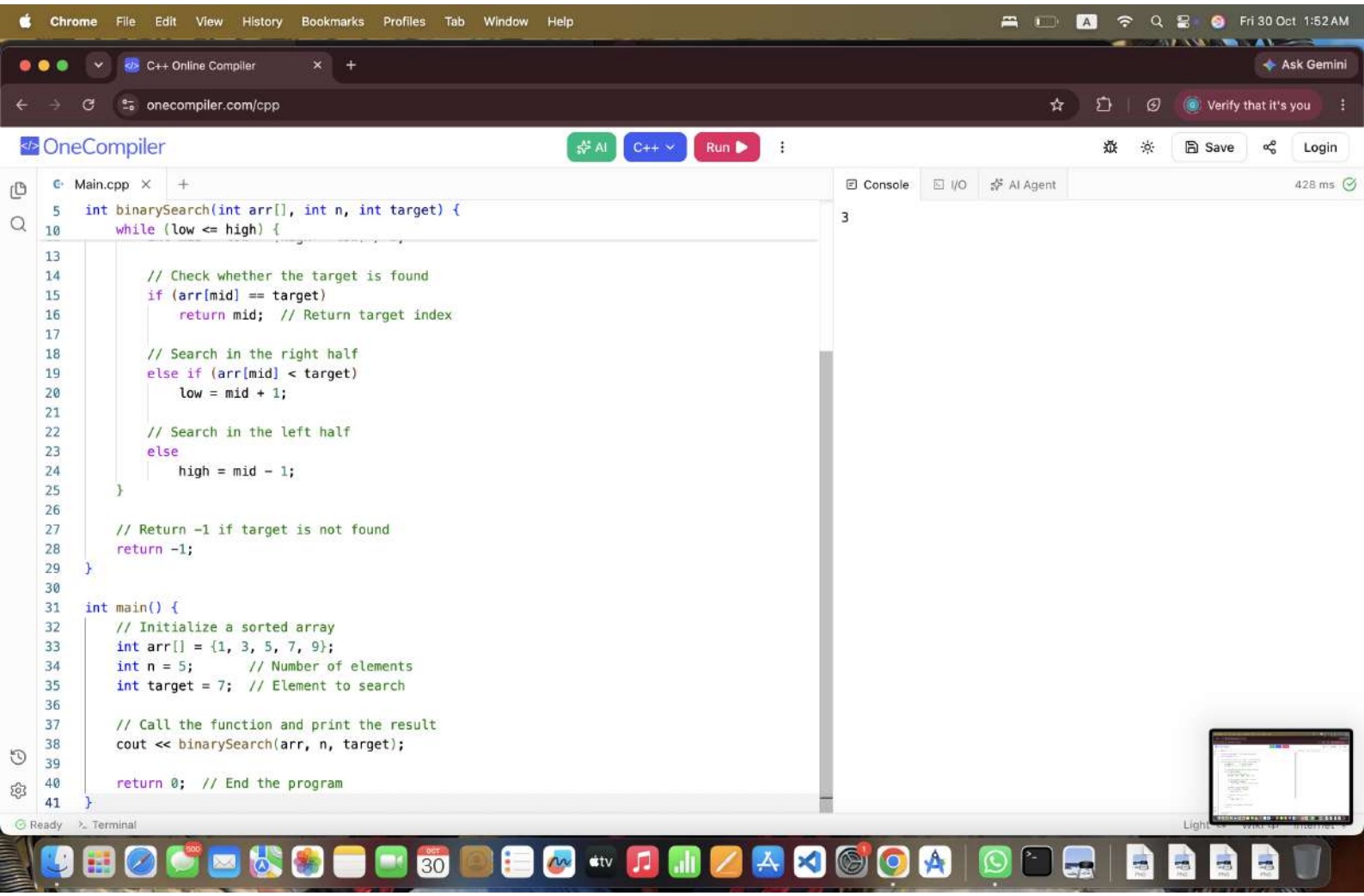
General learning objectives

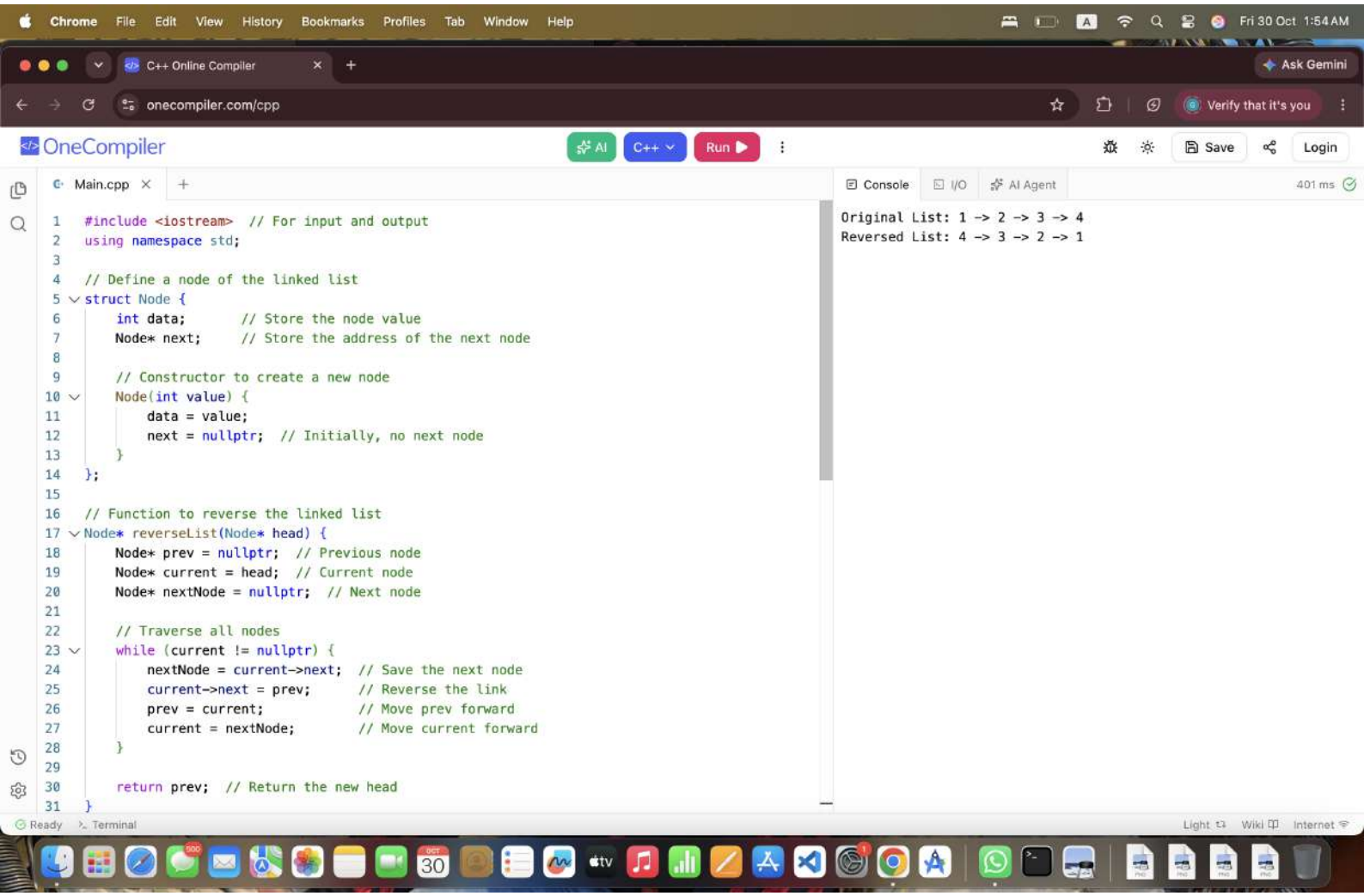
- Understand and apply basic data structures and algorithms to solve programming problems.
- Write readable C++ programs using comments and functions.
- Analyze the time and auxiliary-space complexity of each solution.
- Test solutions using sample inputs and interpret their outputs.

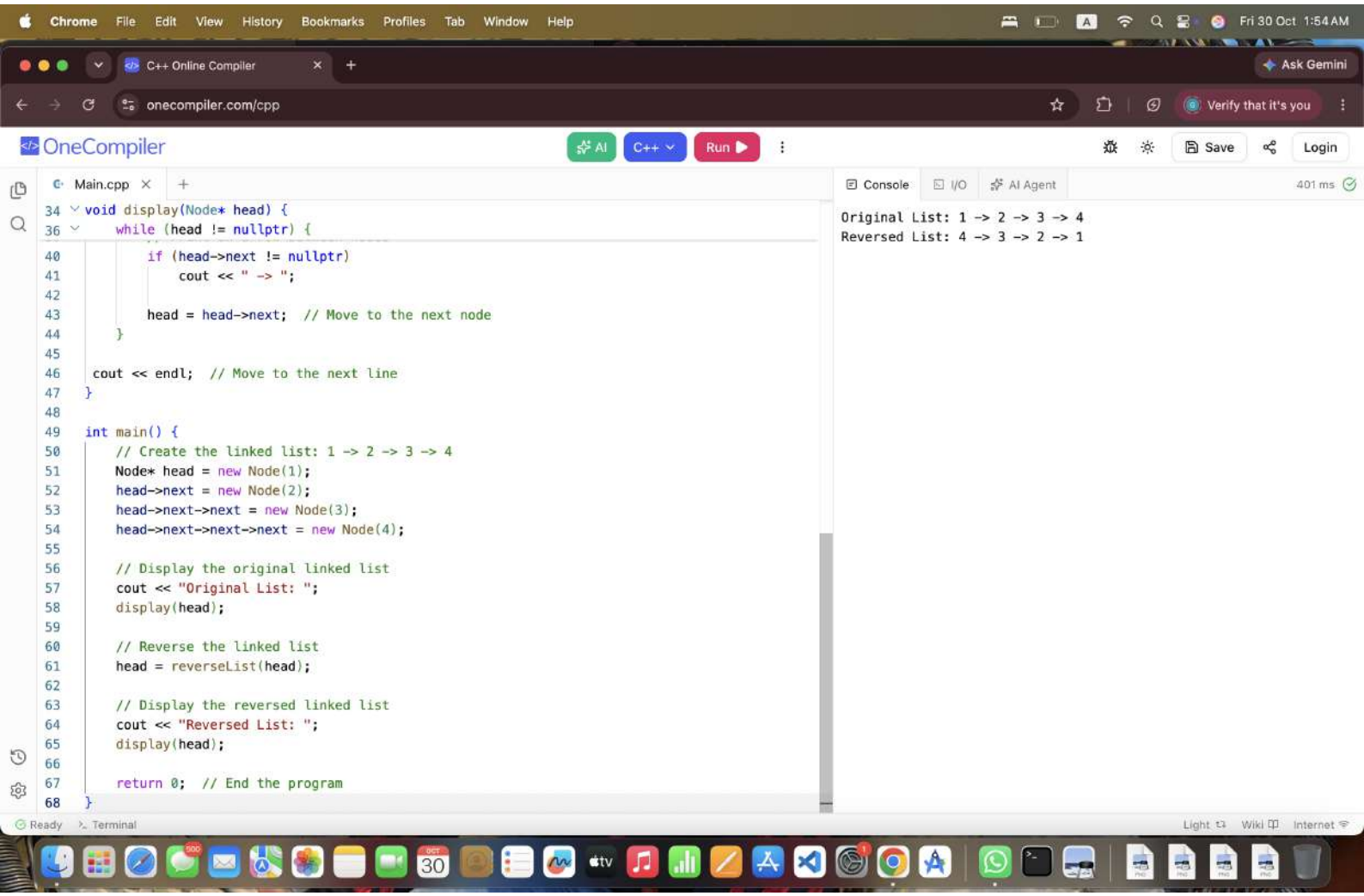
How to run the programs

Copy one question's code at a time into a C++ file in VS Code, for example main.cpp. Compile and run it using your usual C++ compiler. The sample outputs below are expected outputs; they are not screenshots captured from VS Code.


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Fri 30 Oct 1:52 AM
C++ Online Compiler
Ask Gemini
onecompiler.com/cpp
Verify that it's you
OneCompiler
AI
C++
Run
Save
Login
Main.cpp
Console
I/O
AI Agent
428 ms
3
#include <iostream> // For input and output
using namespace std;
// Function to search the target in a sorted array
int binarySearch(int arr[], int n, int target) {
int low = 0; // Starting index
int high = n - 1; // Ending index
// Continue while the search range is valid
while (low <= high) {
// Find the middle index
int mid = low + (high - low) / 2;
// Check whether the target is found
if (arr[mid] == target)
return mid; // Return target index
// Search in the right half
else if (arr[mid] < target)
low = mid + 1;
// Search in the left half
else
high = mid - 1;
}
// Return -1 if target is not found
return -1;
}
int main() {
Ready
Terminal
Light
Wiki
Internet


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Fri 30 Oct 1:52 AM
C++ Online Compiler
Ask Gemini
onecompiler.com/cpp
Verify that it's you
OneCompiler
AI
C++
Run
Save
Login
Main.cpp
Console
I/O
AI Agent
428 ms
3
int binarySearch(int arr[], int n, int target) {
while (low <= high) {
// Check whether the target is found
if (arr[mid] == target)
return mid; // Return target index
// Search in the right half
else if (arr[mid] < target)
low = mid + 1;
// Search in the left half
else
high = mid - 1;
}
// Return -1 if target is not found
return -1;
}
int main() {
// Initialize a sorted array
int arr[] = {1, 3, 5, 7, 9};
int n = 5; // Number of elements
int target = 7; // Element to search
// Call the function and print the result
cout << binarySearch(arr, n, target);
return 0; // End the program
}
Ready
Terminal


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Fri 30 Oct 1:54 AM
C++ Online Compiler
Ask Gemini
onecompiler.com/cpp
Verify that it's you
OneCompiler
AI
C++
Run
Save
Login
Main.cpp
Console
I/O
AI Agent
401 ms
#include <iostream> // For input and output
using namespace std;
// Define a node of the linked list
struct Node {
int data; // Store the node value
Node* next; // Store the address of the next node
// Constructor to create a new node
Node(int value) {
data = value;
next = nullptr; // Initially, no next node
}
};
// Function to reverse the linked list
Node* reverseList(Node* head) {
Node* prev = nullptr; // Previous node
Node* current = head; // Current node
Node* nextNode = nullptr; // Next node
// Traverse all nodes
while (current != nullptr) {
nextNode = current->next; // Save the next node
current->next = prev; // Reverse the link
prev = current; // Move prev forward
current = nextNode; // Move current forward
}
return prev; // Return the new head
}
Original List: 1 -> 2 -> 3 -> 4
Reversed List: 4 -> 3 -> 2 -> 1
Ready
Terminal
Light
Wiki
Internet




```cpp
void display(Node* head) {
    while (head != nullptr) {
        if (head->next != nullptr)
            cout << " -> ";

        head = head->next;  // Move to the next node
    }

cout << endl;  // Move to the next line
}

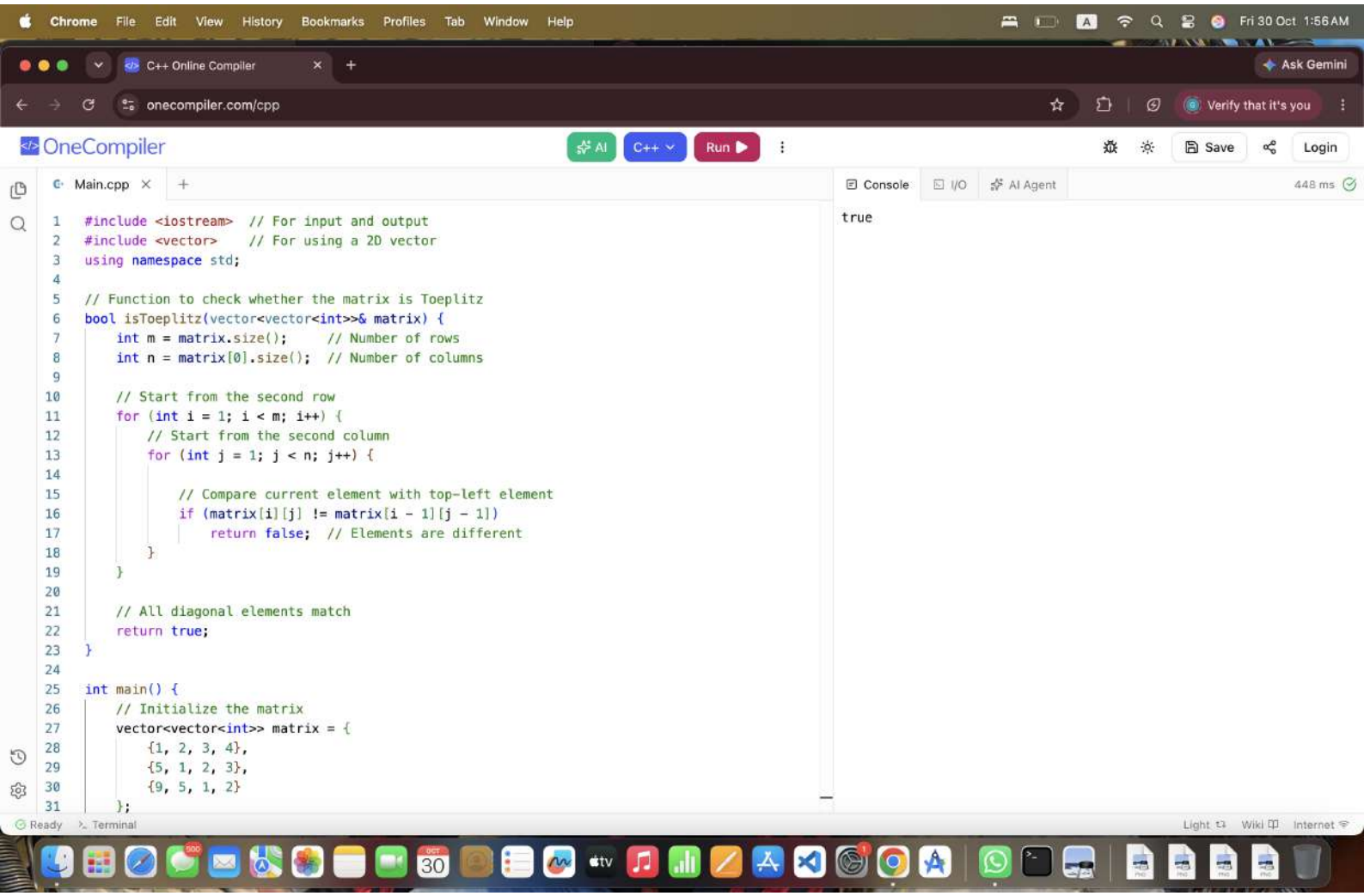
int main() {
    // Create the linked list: 1 -> 2 -> 3 -> 4
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);

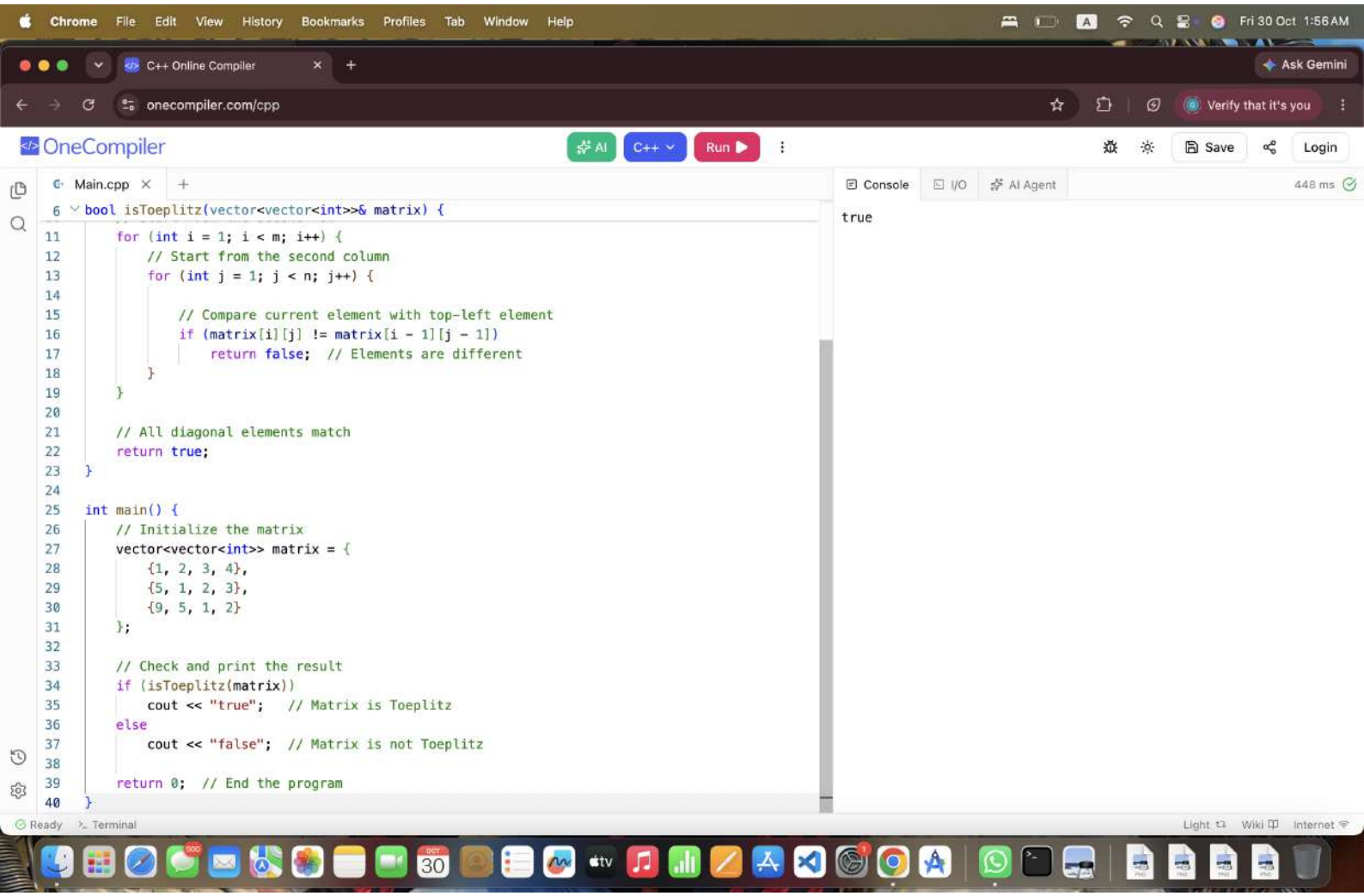
    // Display the original linked list
    cout << "Original List: ";
    display(head);

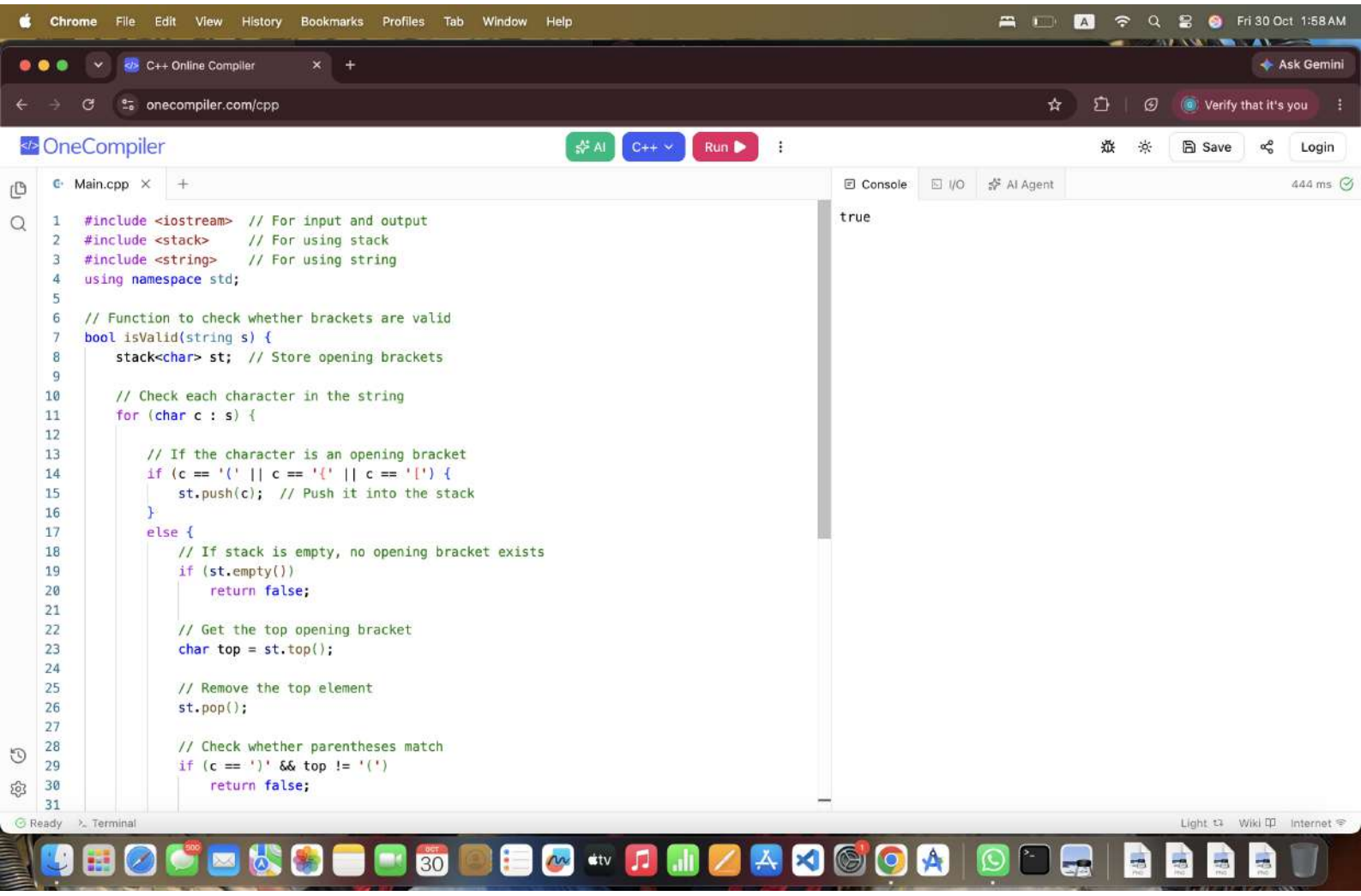
    // Reverse the linked list
    head = reverseList(head);

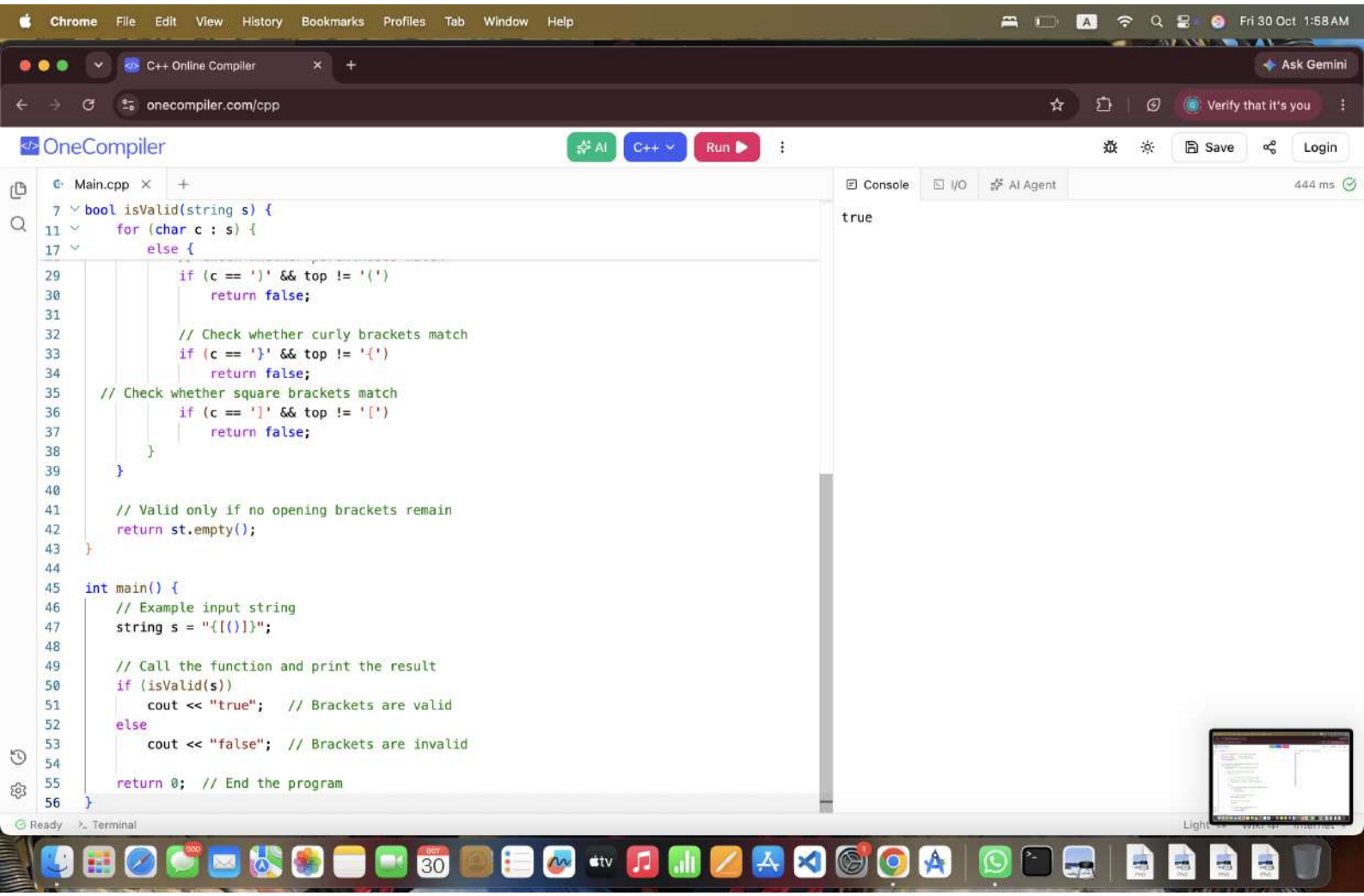
    // Display the reversed linked list
    cout << "Reversed List: ";
    display(head);

    return 0;  // End the program
}
```


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Fri 30 Oct 1:56 AM
C++ Online Compiler
Ask Gemini
onecompiler.com/cpp
Verify that it's you
OneCompiler
AI
C++
Run
Save
Login
Main.cpp
Console
I/O
AI Agent
448 ms
true
#include <iostream> // For input and output
#include <vector> // For using a 2D vector
using namespace std;
// Function to check whether the matrix is Toeplitz
bool isToeplitz(vector<vector<int>>& matrix) {
int m = matrix.size(); // Number of rows
int n = matrix[0].size(); // Number of columns
// Start from the second row
for (int i = 1; i < m; i++) {
// Start from the second column
for (int j = 1; j < n; j++) {
// Compare current element with top-left element
if (matrix[i][j] != matrix[i - 1][j - 1])
return false; // Elements are different
}
}
// All diagonal elements match
return true;
}
int main() {
// Initialize the matrix
vector<vector<int>> matrix = {
{1, 2, 3, 4},
{5, 1, 2, 3},
{9, 5, 1, 2}
};
Ready
Terminal
Light
Wiki
Internet


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Fri 30 Oct 1:56 AM
C++ Online Compiler
Ask Gemini
onecompiler.com/cpp
Verify that it's you
OneCompiler
AI
C++
Run
Save
Login
Main.cpp
Console
I/O
AI Agent
448 ms
true
bool isToeplitz(vector<vector<int>>& matrix) {
for (int i = 1; i < m; i++) {
// Start from the second column
for (int j = 1; j < n; j++) {
// Compare current element with top-left element
if (matrix[i][j] != matrix[i - 1][j - 1])
return false; // Elements are different
}
}
// All diagonal elements match
return true;
}
int main() {
// Initialize the matrix
vector<vector<int>> matrix = {
{1, 2, 3, 4},
{5, 1, 2, 3},
{9, 5, 1, 2}
};
// Check and print the result
if (isToeplitz(matrix))
cout << "true"; // Matrix is Toeplitz
else
cout << "false"; // Matrix is not Toeplitz
return 0; // End the program
}
Ready
Terminal
Light
Wiki
Internet


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Fri 30 Oct 1:58 AM
C++ Online Compiler
Ask Gemini
onecompiler.com/cpp
Verify that it's you
OneCompiler
AI
C++
Run
Save
Login
Main.cpp
Console
I/O
AI Agent
444 ms
true
#include <iostream> // For input and output
#include <stack> // For using stack
#include <string> // For using string
using namespace std;
// Function to check whether brackets are valid
bool isValid(string s) {
stack<char> st; // Store opening brackets
// Check each character in the string
for (char c : s) {
// If the character is an opening bracket
if (c == '(' || c == '{' || c == '[') {
st.push(c); // Push it into the stack
}
else {
// If stack is empty, no opening bracket exists
if (st.empty())
return false;
// Get the top opening bracket
char top = st.top();
// Remove the top element
st.pop();
// Check whether parentheses match
if (c == ')' && top != '(')
return false;
Ready
Terminal
Light
Wiki
Internet


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Fri 30 Oct 1:58 AM
C++ Online Compiler
Ask Gemini
onecompiler.com/cpp
Verify that it's you
OneCompiler
AI
C++
Run
Save
Login
Main.cpp
Console
I/O
AI Agent
444 ms
true
bool isValid(string s) {
for (char c : s) {
else {
if (c == ')' && top != '(')
return false;
// Check whether curly brackets match
if (c == '}' && top != '{')
return false;
// Check whether square brackets match
if (c == ']' && top != '[')
return false;
}
}
// Valid only if no opening brackets remain
return st.empty();
}
int main() {
// Example input string
string s = "{[()]}";
// Call the function and print the result
if (isValid(s))
cout << "true"; // Brackets are valid
else
cout << "false"; // Brackets are invalid
return 0; // End the program
}
Ready
Terminal

DSA LAB ASSIGNMENT

JUSTIFICATION FILE (CO, PO AND KP)

Student Name: _____

Student ID: _____

Course / Section: _____

Questions: Binary Search, Reverse Linked List, Toeplitz Matrix, Valid Parentheses

1. Course Outcomes (CO)

No.	Course Outcome (CO)
CO1	Demonstrate knowledge of fundamental data structures and algorithms.
CO2	Implement fundamental data structures and algorithms using programming tools to solve specified computational problems.
CO3	Analyze problem requirements and justify the selection of appropriate data structures and algorithms.
CO4	Design and develop algorithmic solutions for specified problems using appropriate data structures and algorithms.

2. Program Outcomes (PO)

No.	Program Outcome (PO)
PO(a)	**Engineering Knowledge:** Apply computing and mathematical fundamentals to solve a problem.
PO(b)	**Problem Analysis:** Identify and analyze a computational problem to reach a logical solution.
PO(c)	**Design/Development of Solutions:** Design and implement a solution that meets the stated requirements.
PO(e)	**Modern Tool Usage:** Use suitable tools, such as a code editor, compiler and terminal, to implement and test a solution.

3. Knowledge Profile (KP)

KP	Meaning and relevance
K2 – Conceptual-Based Mathematics	Logical comparisons, index calculations, matrix positions and bracket-matching rules.
K3 – Engineering Fundamentals	Fundamental concepts of arrays, linked lists, matrices, stacks, loops and pointers.
K4 – Specialist Knowledge	Specific knowledge required for binary search, pointer reversal, diagonal comparison and stack-based validation.
K5 – Engineering Design	Planning the algorithm steps and selecting a suitable structure to meet each problem's requirements.
K6 – Engineering Practice	Writing, compiling, running and testing C++ programs using programming tools.

4. CO–PO Mapping Table

Question	CO1	CO2	CO3	CO4	PO(a)	PO(b)	PO(c)	PO(e)
Q1. Binary Search	✓	✓	✓	✓	✓	✓	✓	✓
Q2. Reverse Linked List	✓	✓	✓	✓	✓	✓	✓	✓
Q3. Toeplitz Matrix	✓	✓	✓	✓	✓	✓	✓	✓
Q4. Valid Parentheses	✓	✓	✓	✓	✓	✓	✓	✓

All four questions require understanding a fundamental algorithm or data structure (CO1), implementing it in C++ (CO2), analyzing the requirements and choosing a suitable method (CO3), and developing a working solution (CO4). They use computing concepts (PO(a)), logical analysis (PO(b)), algorithm design (PO(c)), and programming tools (PO(e)).

5. Justification of CO1

Relation to the assignment

CO1 focuses on knowledge of fundamental data structures and algorithms. Q1 uses a sorted array and Binary Search. Q2 uses a singly linked list and pointers to reverse node links. Q3 uses a two-dimensional matrix and compares each element with its top-left neighbour. Q4 uses a stack to check whether brackets are correctly matched and ordered.

PO(a) – Engineering Knowledge

The solutions apply computing and mathematical fundamentals: midpoint and index calculations in Binary Search, pointer operations in a linked list, row-column indexing in a matrix, and last-in-first-out stack behaviour for bracket matching.

Knowledge Profile (KP)

K2 supports index calculations and logical comparisons. K3 supports the use of arrays, linked lists, matrices, stacks, loops and pointers. K4 supports understanding why each algorithm is appropriate for its problem.

6. Justification of CO2

Relation to the assignment

CO2 requires implementation using programming tools. Q1 uses a loop and conditional statements; Q2 updates node pointers; Q3 uses nested loops to compare diagonal neighbours; and Q4 uses the C++ stack library to store opening brackets and match closing brackets.

PO(e) – Modern Tool Usage

The programs can be written in VS Code, compiled with a C++ compiler such as g++, and run in the terminal. Example inputs and outputs can be used to test each solution. Actual screenshots should show the student's own code and output.

Knowledge Profile (KP)

K6 – Engineering Practice is relevant because each solution must be written, compiled, executed and tested. K3 is relevant to implementing the required data structures and operations.

7. Justification of CO3

Relation to the assignment

CO3 focuses on analyzing requirements and choosing a suitable method. Binary Search is suitable because the array is sorted and the search range is halved each step. Q2 reverses given list links using three pointers. Q3 compares each cell with its top-left neighbour. Q4 uses a stack because the latest opening bracket must be matched first.

PO(b) – Problem Analysis

Each question is broken into clear conditions: whether the target equals the middle element, whether list traversal has reached the end, whether diagonal neighbours are equal, and whether a closing bracket matches the stack top.

Knowledge Profile (KP)

K2 supports logical comparisons and conditions. K4 supports selecting a suitable algorithm or data structure. Complexity analysis helps explain the efficiency of the chosen solutions.

8. Justification of CO4

Relation to the assignment

CO4 focuses on designing and developing algorithmic solutions. Q1 updates lower and upper search bounds; Q2 changes each node's next pointer; Q3 checks the required matrix cells; and Q4 pushes opening brackets and checks each closing bracket against the stack top. Each program prints the required result.

PO(c) – Design/Development of Solutions

Each program is designed around the problem requirements with suitable variables, control flow, data structures and output. Running example cases helps verify that the solution works as expected.

Knowledge Profile (KP)

K5 – Engineering Design applies to planning the algorithm and selecting a suitable structure. K6 – Engineering Practice applies to coding, compiling, running and checking the output.

9. Algorithm Efficiency

Question	Data structure / method	Time	Auxiliary space
Q1. Binary Search	Sorted array; halve search range	O(log n)	O(1)
Q2. Reverse Linked List	Linked list; three pointers	O(n)	O(1)
Q3. Toeplitz Matrix	Matrix; compare top-left neighbours	O(m × n)	O(1) extra
Q4. Valid Parentheses	Stack; match brackets	O(n)	O(n)

Here, n is the number of array elements, linked-list nodes or string characters, as applicable. For the matrix, m and n represent its row and column counts. Auxiliary space excludes the input data itself.

10. Conclusion

This assignment connects the four programming questions with the proposed course and program outcomes. The questions demonstrate fundamental algorithm and data-structure knowledge, implementation in C++, analysis of problem requirements, design of working solutions, and practice in testing and evaluating complexity.

Note: The CO, PO and KP wording is proposed and adapted to the sample format you provided. If your instructor or department has an official CO/PO/KP list, replace these descriptions and mappings with the official version.